

[Bài đọc] Xây dựng 1 dự án với Spring boot

1. Spring Boot là gì?

- Spring Boot là một framework dựa trên **Spring** giúp việc xây dựng ứng dụng Java trở nên nhanh chóng và dễ dàng hơn nhờ:
 - **Tự động cấu hình (Auto Configuration):** giảm việc viết file cấu hình rườm rà
 - **Embedded Server:** tích hợp sẵn Tomcat/Jetty/Undertow, không cần cài đặt thủ công
 - **Starter Dependencies:** gói phụ thuộc đã được cấu hình sẵn
 - **Sẵn sàng cho production:** hỗ trợ logging, giám sát, health-check
- Mục tiêu chính: **Giảm cấu hình, tập trung vào logic nghiệp vụ**

2. Chuẩn bị môi trường

- Trước khi bắt đầu, bạn cần:
 - **Java Development Kit (JDK):** Phiên bản 17 hoặc mới hơn
 - **IDE:** IntelliJ IDEA, Eclipse hoặc VS Code
 - **Maven hoặc Gradle:** Spring Boot hỗ trợ cả hai
 - **Postman** (nếu làm API)
 - **Git** (nếu cần quản lý code)

3. Quy trình xây dựng một dự án Spring Boot

- Bước 1: Xác định yêu cầu
 - Xác định **mục tiêu:** web app, REST API, microservice,...
 - Liệt kê các **tính năng:** CRUD, phân quyền, kết nối CSDL,...

- Chọn **CSDL**: MySQL, PostgreSQL, MongoDB, v.v
- Bước 2: Khởi tạo project
 - Link hướng dẫn: https://tuan-binh.github.io/tutorial.github.io/Tao_du_an_Spring_Boot.p
- Bước 3: Cấu trúc thư mục chuẩn

```
src/main/java
    com.example.demo
        DemoApplication.java        // File main chạy Spring Boot
        controller/                 // Các controller xử lý request
        service/                    // Xử lý logic nghiệp vụ
        repository/                 // Tương tác DB
        model/                      // Các class entity
src/main/resources
    application.properties or application.yml // File cấu hình
    static/                          // Chứa file tĩnh: CSS, JS, images
    templates/                      // Chứa view (Thymeleaf, JSP,...)
```

- Bước 4: Cấu hình dự án
 - Trong **application.properties** hoặc **application.yml**:

```
spring.datasource.url=jdbc:mysql://localhost:3306/demo_db
spring.datasource.username=root
spring.datasource.password=123456
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
server.port=8080
```

- **ddl-auto=update**: Tự tạo bảng nếu chưa tồn tại
 - **show-sql=true**: Hiển thị câu SQL khi chạy

- Bước 5: Viết Entity (Model)

```
@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private double price;

    // getters & setters
}
```

- Bước 6: Tạo Repository

```
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public class ProductRepository {

    @PersistenceContext
    private EntityManager entityManager;

    public List<Product> findAll() {
        String jpql = "SELECT p FROM Product p";
        return entityManager.createQuery(jpql, Product.class).getResultList();
    }

    public Product findById(Long id) {
        return entityManager.find(Product.class, id);
    }

    public Product save(Product product) {
        if (product.getId() == null) {
            entityManager.persist(product); // Thêm mới
            return product;
        } else {
            return entityManager.merge(product); // Cập nhật
        }
    }

    public void deleteById(Long id) {
        Product product = findById(id);
        if (product != null) {
            entityManager.remove(product);
        }
    }
}
```

- Bước 7: Tạo Service

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class ProductService {

    @Autowired
    private ProductRepository repo;

    public List<Product> findAll() {
        return repo.findAll();
    }

    public Product save(Product p) {
        return repo.save(p);
    }

    public void delete(Long id) {
        repo.deleteById(id);
    }
}
```

- Bước 8: Tạo Controller

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/products")
public class ProductController {

    @Autowired
    private ProductService service;

    // Hiển thị danh sách sản phẩm
    @GetMapping
    public String listProducts(Model model) {
        model.addAttribute("products", service.findAll());
        return "product-list"; // templates/product-list.html
    }

    // Hiển thị form tạo mới sản phẩm
    @GetMapping("/new")
    public String showCreateForm(Model model) {
        model.addAttribute("product", new Product());
        return "product-form"; // templates/product-form.html
    }

    // Xử lý lưu sản phẩm mới
    @PostMapping
    public String saveProduct(@ModelAttribute("product") Product product) {
        service.save(product);
        return "redirect:/products";
    }

    // Xóa sản phẩm
    @GetMapping("/delete/{id}")
    public String deleteProduct(@PathVariable Long id) {
        service.delete(id);
        return "redirect:/products";
    }
}
```

- Bước 9: Tạo view
 - product-list.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Danh sách sản phẩm</title>
</head>
<body>
<h1>Danh sách sản phẩm</h1>
<a th:href="@{/products/new}">Thêm sản phẩm</a>
<table border="1">
    <tr>
        <th>ID</th>
        <th>Tên sản phẩm</th>
        <th>Giá</th>
        <th>Hành động</th>
    </tr>
    <tr th:each="p : ${products}">
        <td th:text="${p.id}"></td>
        <td th:text="${p.name}"></td>
        <td th:text="${p.price}"></td>
        <td>
            <a th:href="@{'/products/delete/' + ${p.id}}">Xóa</a>
        </td>
    </tr>
</table>
</body>
</html>

```

- product-form.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Thêm sản phẩm</title>
</head>
<body>
<h1>Thêm sản phẩm</h1>
<form th:action="@{/products}" th:object="${product}" method="post">
    <label>Tên sản phẩm:</label>
    <input type="text" th:field="*{name}" required>
    <br>
    <label>Giá:</label>
    <input type="number" step="0.01" th:field="*{price}" required>
    <br>
    <button type="submit">Lưu</button>
</form>
<a th:href="@{/products}">Quay lại</a>
</body>
</html>

```

- Bước 10: Chạy thử
 - Chạy file `DemoApplication.java` (có `@SpringBootApplication`)
 - Mở trình duyệt: <http://localhost:8080/products>

4. Mẹo và lưu ý

- Dùng Lombok (`@Data`, `@Builder`) để giảm code getter/setter

- Quản lý version dependency qua `spring-boot-starter-parent`
- Sử dụng `application.yml` thay vì `.properties` cho cấu hình phức tạp
- Khi triển khai thật, dùng `spring.profiles.active=prod` để phân môi trường

Danh sách các bài học

